

HACKNEX 2026

National Hackathon · Problem Statements

Agentic AI

HNX26EPS01: Agentic Legal Assistant

Pitch: An agentic platform that reasons over legal case documents and supports contract/case review, legal drafting, legal research, and grounded RAG chat, where every fact and citation is verifiable.

The four workflows:

1. **Contract / Case Review:** read and reason over case documents; identify key facts, contradictions, missing information, and relevant evidence with source references.
2. **Legal Drafting:** draft bail applications, petitions, notices, affidavits, etc. *without inventing facts or citations*.
3. **Legal Research:** research relevant laws, judgments, and precedents; connect them to the case; provide verified sources and citations.
4. **RAG Chat:** grounded conversational Q&A over retrieved documents, maintaining context.

Why it's hard: The bar is **verifiability, not fluency**. Every drafted fact and every cited law/judgment must be traced to a retrieved source. A **fabricated** fact or citation (one that does not exist in the sources) is a hard failure; a true-but-unsupported claim lowers the groundedness score. This turns "did it sound good" into "can you prove groundedness."

Baseline to beat (chosen by the team): The strongest option of this kind available today, chosen by the team (for example a retrieval-augmented pipeline on a leading general-purpose LLM), run on the same inputs.

Minimum bar to qualify (24h): A working system on one or more of the four workflows that beats the baseline on groundedness on the held-out queries, with no fabricated facts or citations, plus a short write-up. Teams choose their own scope; depth on one workflow counts for more than shallow coverage of all four.

Stretch goals:

- A contradiction-detection pass across documents.
- A "confidence + missing-info" report the agent surfaces *before* drafting.
- Second and subsequent workflows.

Data and judging: Teams source their own public case files and statutes. At judging, judges bring unseen documents and queries with known correct source references.

Judging rubric:

- Groundedness vs. baseline: % of claims traceable to a real source (35%)
- Zero fabricated facts / citations (pass/fail gate)
- Retrieval quality vs. baseline (20%)

- Usefulness of the drafted / answered output (20%)
- Research contribution: what is new compared with the baseline, shown by an ablation or comparison (25%)
- *Bonus*: contradiction detection, missing-info reporting, additional workflows

Skills: RAG, agent orchestration, retrieval eval, prompt/grounding design.

Generative AI

HNX26EPS02: Realtime Multilingual Conversational Bot (Indian Languages)

Pitch: A realtime speech-in / speech-out conversational bot that supports the Dravidian languages (**Tamil, Telugu, Kannada, Malayalam**) plus **Hindi** and **English**, with low latency and natural responses.

Why it's hard: Off-the-shelf model support for these languages is patchy compared to English, so teams have to be clever about which models/APIs they stitch together. The real difficulty is **code-switching**: people constantly mix English words into Tamil/Telugu/etc. sentences mid-utterance. On top of that come on-the-fly language identification, low-resource ASR/TTS quality, and keeping the whole loop realtime.

Baseline to beat (chosen by the team): The strongest option of this kind available today, chosen by the team (for example the best commercial speech APIs or open models stitched together for these languages), run on the same inputs.

Minimum bar to qualify (24h): A working speech → understanding → speech loop in English plus at least **2–3 Indic languages**, with automatic language detection, that beats the baseline on latency (time from the end of the user's speech to the start of the bot's audio) without losing accuracy, plus a short write-up.

Stretch goals:

- All six languages.
- Seamless code-switch handling within a single utterance.
- On-device / offline operation.
- Consistent persona and context memory across turns.

Data and judging: Teams choose their own models, APIs and test material. At judging, judges speak unseen utterances in each language, including code-switched ones.

Judging rubric:

- Latency vs. baseline: end of speech to start of the bot's audio (25%)
- ASR accuracy per language vs. baseline (20%)
- Language identification and code-switch handling vs. baseline (25%)
- TTS naturalness (10%)
- Research contribution: what is new compared with the baseline, shown by an ablation or comparison (20%)
- *Bonus*: full six-language coverage, offline operation

Skills: multilingual ASR/TTS, language ID, low-resource NLP, latency engineering.

HNX26EPS03: Live Translation for Indic Languages

Pitch: Build a **live translation** system for **Hindi, Tamil, Telugu, Kannada, Malayalam and English**. One person speaks in one language and the listener reads (or hears) the translation in another, as the speech happens. Speech output is optional, and teams that choose it may also take on **voice cloning**: the translated speech is spoken in the original speaker's own voice. There are no restrictions on how teams build it: APIs, open models, their own models, or any mix of them, in the cloud or locally.

Scope: This track is about translating *between people* in real time. Building an AI that holds a conversation with the user is a different task (see HNX26EPS02) and is not what is judged here.

Why it's hard: Real-time translation is not just "ASR, then MT, then TTS" run in a row. The system has to decide **when to commit to a translation before the speaker has finished the sentence**, and Indic languages make this harder: Hindi and the Dravidian languages put verbs at the end, where English puts them in the middle, so the meaning often isn't known until late. On top of that, errors in speech recognition carry into translation, people mix English into Tamil / Telugu / Hindi mid-sentence, and names, numbers and domain terms are easy to get wrong. Plugging in good APIs is allowed and expected; the winning work is in **orchestration**: segmentation, partial results, correction, and keeping latency low without losing accuracy. Teams that attempt voice cloning face an extra problem: the cloned voice has to carry over into a language the speaker never spoke in, and stay natural and intelligible at streaming latency.

Baseline to beat (chosen by the team): The strongest option of this kind available today, chosen by the team (for example the best commercial live-translation service or open models), run on the same inputs.

Minimum bar to qualify (24h): A working live demo with microphone input and translated output (text captions are enough) between English and at least **2–3 Indic languages** that beats the baseline on latency without losing translation quality on held-out utterances, plus a short write-up.

Stretch goals:

- All six languages, translating between any pair (not only via English).
- **Voice cloning:** the translated speech is spoken in the original speaker's own voice (cross-lingual voice cloning from a short sample), keeping identity and natural expression. It is done only with the speaker's consent and is clearly disclosed as synthetic.
- Speech-to-speech output with natural-sounding voices.
- Two-way conversation mode: automatic language detection and speaker turns, so two people can each speak their own language.
- Stable live captions: partial translations that update smoothly instead of flickering or rewriting themselves.
- Custom glossary support for names, places and domain terms, plus handling of code-mixed speech.

Data and judging: Teams choose their own models, APIs and test material. At judging, judges speak unseen utterances per language pair (including code-mixed sentences, names and numbers) and compare against reference translations.

Judging rubric:

- Translation quality per language pair vs. baseline (chrF++ / BLEU, plus human or model-based adequacy checks) (25%)
- Latency vs. baseline: time to first translated word and delay from end of speech to final translation (25%)
- Streaming quality: caption stability (few late rewrites) and correct sentence boundaries (15%)
- Language detection and robustness to code-mixing, names, numbers, accents and noise (15%)
- Research contribution: what is new compared with the baseline, shown by an ablation or comparison (20%)
- *Bonus*: all six languages, any-pair translation, speech output, voice cloning (speaker similarity, naturalness and intelligibility of the cloned translated speech, judged on a consenting speaker), two-way conversation, glossary

Skills: streaming ASR, machine translation, TTS, voice cloning / speaker embeddings, language identification, API / model orchestration, latency engineering.

HNX26EPS04: Extreme Bad-Handwriting Digitizing Stack

Pitch: A model / agent / software stack that reliably digitizes *genuinely bad* handwriting (doctor's-scrrawl level: cramped, inconsistent, crossed-out words, margin notes, mixed scripts) into clean, editable text.

Why it's hard: Standard OCR/HTR is trained on neat data and falls apart on real messy handwriting. The intended clever solution is a *pipeline*, not a single model: combine handwriting recognition with a vision-language model and language-model post-correction (using context to fix garbled output), handle layout, and, crucially, **flag uncertainty instead of hallucinating** confident wrong text.

Baseline to beat (chosen by the team): The strongest option of this kind available today, chosen by the team (for example leading OCR / handwriting recognition and a general-purpose vision-language model), run on the same inputs.

Minimum bar to qualify (24h): A pipeline that takes bad-handwriting images and outputs editable text, beats the baseline on held-out hard samples, and explicitly flags illegible regions instead of guessing, plus a short write-up.

Stretch goals:

- Multilingual / mixed-script handwriting.
- Structure preservation (tables, forms, layout).
- Diagram / sketch handling.
- Human-in-the-loop correction UI.

Data and judging: Teams collect their own handwriting samples and transcriptions for development. At judging, judges bring unseen, genuinely hard samples (not neat cursive) with ground-truth transcriptions.

Judging rubric:

- Character / word accuracy on held-out *hard* samples vs. baseline (40%)
- Uncertainty flagging: errors are flagged rather than confidently wrong (25%)
- Robustness to crossed-out text, margin notes, poor image quality (15%)
- Research contribution: what is new compared with the baseline, shown by an ablation or comparison (20%)
- *Bonus:* mixed-script, structure preservation, correction UI

Skills: OCR/HTR, vision-language models, LLM post-correction, layout analysis.

Computer Vision

HNX26EPS05: Multi-Stream Video Intelligence with Conversational Query

Pitch: Plug in multiple CCTV streams; the system continuously detects and indexes events across all cameras, and exposes a chat interface where a user describes what they want ("did a red car pass through the main gate in the last hour?") and gets back *which camera, when*, and the visual evidence.

Why it's hard: Four separable challenges:

- **Open-vocabulary search.** Queries like "red car" or "person carrying a large bag" won't match a fixed label set. This needs open-vocab detection or vision-language embeddings, not a closed class list.
- **Grounded, localized answers.** Every answer resolves to a specific camera + timestamp + visual evidence. No traceable source = scored failure.
- **Clarify-once, then remember.** When the system doesn't know a referent ("which one is the main gate?"), it asks, lets the user point to the camera/region, and then **persists that fact permanently**. It must never ask again, even after a restart. This is a small learned knowledge base, not just chat history.
- **Cross-camera continuity.** Re-identify the same car/person across non-overlapping cameras and reconstruct its path ("Gate cam 09:14 → lobby cam 09:16 → rear cam 09:22").

Baseline to beat (chosen by the team): The strongest option of this kind available today, chosen by the team (for example a standard open-vocabulary detection plus embedding-retrieval pipeline), run on the same footage.

Minimum bar to qualify (24h): Use **recorded** multi-camera footage (not live). Support natural-language queries that return the correct camera + timestamp + clip, beating the baseline on retrieval accuracy for the held-out queries, with the clarify-once memory behavior, plus a short write-up.

Stretch goals:

- True live multi-stream ingestion.
- Cross-camera re-ID and timeline reconstruction.
- Standing queries / real-time alerts ("notify me if anyone enters this zone after 8pm").
- Privacy handling (face blurring, on-prem-only inference).

Data and judging: Teams use their own multi-camera footage, recorded or taken from public datasets. At judging, judges bring unseen recorded multi-camera footage and natural-language queries with known answers.

Judging rubric:

- Retrieval accuracy on held-out NL queries vs. baseline (30%)
- Correctness of camera + timestamp localization (20%)
- Clarify-once memory: asks appropriately, never re-asks, **persists across restart** (20%)

- Query latency vs. baseline (10%)
- Research contribution: what is new compared with the baseline, shown by an ablation or comparison (20%)
- *Bonus*: live ingestion, cross-camera tracking, alerts, privacy

Skills: video decoding pipelines, open-vocab / VLM detection, embedding retrieval + temporal indexing, multi-object tracking + re-ID, persistent memory design.

HNX26EPS06: 3D Scene Generation from Blueprints and Room Video

Pitch: Turn a 2D description of a static space into a navigable 3D model. The input is either an architectural blueprint / floor plan, or ordinary images and video of a real room. The system outputs a 3D representation of the space and, for video input, **generates the parts that were never seen** (the wall behind the sofa, the ceiling, the corner nobody filmed) instead of leaving holes.

Scope (static spaces only): This track is about **static** environments: a room or building that does not change while it is captured. Anything that moves, and any time-based playback, belongs to HNX26EPS07 (4D) and is out of scope here. Entries are judged only on how well they reconstruct and complete the space itself.

Why it's hard: Reconstruction from what the camera saw is a solved-ish problem; **completion of what it did not see** is the research frontier. Teams have to recover metric scale and consistent geometry from messy handheld footage, parse symbolic blueprints (walls, doors, windows, dimensions) into real structure, and then fill unobserved regions plausibly while being honest about which parts are observed and which are generated.

Baseline to beat (chosen by the team): The strongest option of this kind available today, chosen by the team for the mode it enters (for example an off-the-shelf floor-plan parser for Mode A, or a standard structure-from-motion plus Gaussian splatting / NeRF pipeline for Mode B), run on the same inputs.

Minimum bar to qualify (24h): Pick **one** input mode and do it end to end, beating the baseline for that mode on the held-out inputs. Either (a) a floor plan → a 3D model with correctly placed walls, doors, windows and room dimensions, or (b) a short walkthrough video of a static room → a 3D scene (mesh, point cloud or Gaussian splat) that can be orbited in a viewer, with at least one visibly completed unseen region. The output must open in a real 3D viewer. A short write-up is also required.

Stretch goals:

- Both input modes in one pipeline (blueprint-guided completion of a video reconstruction).
- Clear visual distinction between observed geometry and generated geometry (confidence map or colour overlay).
- Furniture / object-level reconstruction, not just the room shell.
- Reconstruction from a sparse set of photos instead of a full video.
- Exporting to a standard format (GLB / PLY / USD) usable in common 3D tools.

Data and judging: Teams capture or find their own room videos and floor plans. At judging, judges bring unseen floor plans and room videos with measured dimensions, with some camera views held out.

Judging rubric:

Pick one mode. Your entry is scored only with that mode's rubric, and each mode is worth 100 points. Doing both modes in one pipeline is a bonus.

Mode A: Floor plan → 3D model

- Layout accuracy vs. baseline: wall, door and window placement and room dimensions (layout IoU and dimension error) (40%)

- Completeness: every room and opening in the plan is present and correct (15%)
- Model quality: clean, correctly scaled, sensible wall heights and proportions (15%)
- Usability of the output in a 3D viewer (10%)
- Research contribution: what is new compared with the baseline, shown by an ablation or comparison (20%)

Mode B: Room video → 3D scene

- Novel-view quality on held-out camera views vs. baseline (PSNR / SSIM / LPIPS) (20%)
- Geometric accuracy vs. baseline (dimensions, Chamfer distance) (25%)
- Quality and plausibility of generated unseen regions (15%)
- Honesty: observed vs. generated regions are clearly separated (no silently hallucinated geometry) (15%)
- Usability of the output in a 3D viewer (5%)
- Research contribution: what is new compared with the baseline, shown by an ablation or comparison (20%)

Bonus (either mode)

- Both modes in one pipeline, object-level reconstruction, standard-format export

Skills: 3D vision, structure-from-motion, neural rendering / Gaussian splatting, generative completion (diffusion / inpainting), document parsing.

HNX26EPS07: 4D Scene Reconstruction and a Traversable Video Format

Pitch: Take a video and turn it into a **4D representation**: a 3D scene that also lives in time. Instead of watching a clip from the fixed angle the camera happened to have, a viewer steps *into* the moment and moves with **6 degrees of freedom** (any position, any angle) while the people and objects inside keep moving as they did in the footage. The end goal is a **traversable video file**: a compact new file format / codec, plus a lightweight player, so these moments can be shared and played on ordinary end devices *without* a heavy GPU.

Scope (dynamic scenes only): This track is about **moving** scenes: people and objects that change over time. Reconstructing a static room, parsing floor plans, or completing unseen regions of a static space belongs to HNX26EPS06 (3D) and is out of scope here. Entries are judged on how well they handle motion and the delivery of a time-based scene.

Why it's hard: Reconstructing a moving scene is already hard: the system has to separate moving from static content, keep motion consistent over time (no flicker, no melting limbs), and render believable views from camera positions that **never existed in the original footage**. The real difficulty is the **delivery**. Research-grade 4D reconstructions are huge and need a strong GPU to render. A usable format has to compress the representation, decode it quickly, and play it smoothly on a phone or an integrated-graphics laptop. Quality, file size and playback cost all pull against each other.

Baseline to beat (chosen by the team): The strongest option of this kind available today, chosen by the team (for example a standard dynamic Gaussian-splat / neural-rendering reconstruction in its raw exported form). Quality on the held-out view and file size are both measured against it.

Minimum bar to qualify (24h): A **short** clip (a few seconds) from synchronized multi-camera footage, reconstructed as a 4D scene, **exported to a single file**, and played back in a team-built player where the user can scrub through time and freely move the camera, with moving elements playing back correctly. The player must run on a device **without a discrete GPU** (an integrated-graphics laptop or a phone browser), which the judges choose at judging time. The reconstruction is scored on a camera view that was held out and never shown to the team: **judges supply the camera pose, and the team's player must render a frame for that pose from the exported file** (through a command-line option, API or URL parameter, which the team documents). The judges run it on that device. All quality metrics are computed from the decoded file, not from the uncompressed model. The exported file must be smaller than the baseline's at equal or better quality, and the team submits a short write-up.

Stretch goals:

- A real codec: measurable compression of the representation (rate vs. quality), with the format documented well enough for someone else to write a decoder.
- Playback on several different end devices / browsers from the same file, with no discrete GPU.
- Streaming playback, so the clip starts before the whole file has downloaded.
- Monocular input: a single phone video instead of a multi-camera rig.
- Playback controls beyond scrubbing: slow-motion, freeze-frame / "bullet-time" fly-arounds.

Data and judging: Teams use their own multi-camera clips (recorded themselves or from public multi-view datasets), their own compute, and build their own player. Each team holds out one camera view that is never used in training, and judges verify it by supplying a camera pose at judging time and comparing the player's render against that held-out view.

Judging rubric:

- Novel-view quality on the held-out camera, computed from the decoded file, vs. baseline (PSNR / SSIM / LPIPS) (20%)
- Temporal consistency (flicker, motion fidelity, no ghosting) (10%)
- Freedom of navigation: true 6-DoF movement, not just small shifts around the original cameras (10%)
- Playback on a device without a discrete GPU (frame rate and responsiveness) (pass/fail gate for the player)
- File size and compression: quality retained per megabyte vs. the baseline representation (25%)
- Format quality: single self-contained file, documented structure, fast decode (15%)
- Research contribution: what is new compared with the baseline, shown by an ablation or comparison (20%)
- *Bonus*: real codec with rate-distortion curve, multi-device playback, streaming, monocular input

Skills: dynamic 3D reconstruction, neural rendering / Gaussian splatting, video codecs and compression, WebGL / WebGPU or native player development, real-time graphics.

Edge AI

HNX26EPS08: On-Device Conversational Stack

Pitch: Build a full ASR → LLM → TTS conversation loop that runs entirely on-device, with no cloud calls and very low compute usage. Teams develop and demonstrate it on their own laptops. The edge device is simulated by restricting the laptop: CPU-only, with no GPU, and a small slice of its cores and memory enforced through OS or container limits. The smaller the footprint, the better.

Why it's hard: The models aren't the challenge; the budget is. Memory, latency, and power are all against you. Teams have to quantize aggressively, stream where possible, and degrade gracefully when the device is starved.

Baseline to beat (chosen by the team): The strongest option of this kind available today, chosen by the team (for example a standard quantized ASR + small LLM + TTS stack run with default settings under the same resource limit).

Minimum bar to qualify (24h): A working voice-in → voice-out loop, fully offline and CPU-only, running under a resource limit the team declares and shows enforced (for example a set number of cores and amount of RAM), with wake-word / VAD handling. It must beat the baseline on end-to-end latency (end of speech to first audio of the reply) at a lower resource footprint. Teams report measured CPU, RAM and energy use, plus a short write-up.

Stretch goals:

- Multilingual support.
- Run the loop on a smaller device the team owns (phone, Raspberry Pi or similar).
- Cached / pre-synthesized TTS for common responses to cut latency.
- Keep quality as the resource limit is tightened further.

Data and judging: Teams use their own laptops, models and test material. At judging, the judges speak unseen requests and measure CPU, RAM and energy use on the team's own laptop under the declared limit.

Judging rubric:

- On-device inference, no cloud, no GPU (pass/fail gate)
- End-to-end latency vs. baseline (25%)
- Resource footprint vs. baseline: CPU, RAM and energy use under the declared limit (25%)
- Quantization + offload strategy (15%)
- Graceful degradation when the limit is tightened (15%)
- Research contribution: what is new compared with the baseline, shown by an ablation or comparison (20%)
- *Bonus:* multilingual, smaller-device operation, cached TTS

Skills: embedded, quantization, streaming ASR/TTS, latency engineering.

